I-Logical Approaches Propositional Syntax and Tensor Network Decompositions

Foundations of Neuro-Symbolic AI

Alex Goessmann

University of Applied Science Würzburg-Schweinfurt (THWS)

March 23, 2026

Outline

- Syntactical Decomposition of Formulas
- Basis encodings
- Basis calculus
- Implementations in `tnreason`

Decomposition of formulas

So far: Decomposition of a Knowledge base into formulas
Now: Generic syntactical decompositions of formulas

Decomposition into logical connectives

Strategy: Store the semantics of formulas f_1, f_2 and $f_1 \circ f_2$ at each decomposition of the formula.

Distributed Representation

The semantics of complex formulas is then stored by a set of semantics of the used logical connectives $\circ \in \{\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow\}$.

How to represent formulas as tensor networks?

Multilinearity of tensor networks

Tensor networks are multilinear, that is linear in each core tensor decorating an edge.

The logical conjunction is the only connective function which is multilinear.

Non-linearity problem

How to represent non-linear formulas like $\neg X$ in a tensor network?

Solution:
We develop the basis calculus scheme instead of coordinate calculus.

Outline

- Syntactical Decomposition of Formulas
- Basis encodings
- Basis calculus
- Implementations in `tnreason`

Recall: One-hot encodings

One-hot encodings of a state are a tensor with a single coordinate 1 marking the corresponding state.

One-hot encodings

For a state $x \in [m]$ of a variable X the one-hot encoding is

$$\boxed{\epsilon_{x_0}} \quad = \begin{pmatrix} 0 & \cdots & 0 & 1 & 0 & \cdots & 0 \end{pmatrix} [X_0]$$

$$\begin{array}{ccccccc} \vdots & & & \vdots & & & \vdots \\ 0 & & & x_0 & & & m_0 - 1 \end{array}$$

Given a factored representation with multiple variables $X_{[d]}$, the one-hot encoding of a state $x_{[d]}$ is the tensor product

$$\epsilon_{x_{[d]}} \left[X_{[d]}\right] = \bigotimes_{k \in [d]} \epsilon_{x_k} \left[X_k\right] . \tag{1}$$

They build an orthogonal basis of the tensor space.

Coordinate and basis encoding

Coordinate encoding:
Store an information $f(x_{[d]})$ in the coordinates of a tensor

$$f\left[X_{[d]}\right] = \sum_{x_{[d]} \in [2]^d} f(x_{[d]}) \cdot \epsilon_{x_{[d]}}\left[X_{[d]}\right]$$

Basis encoding:
Store an information $f(x_{[d]})$ in a basis vector $\epsilon_{f(x_{[d]})}\left[Y_f\right]$ to an auxiliary variable Y_f, that is

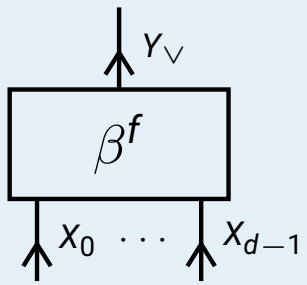
$$\beta^f\left[Y_f, X_{[d]}\right] = \sum_{x_{[d]} \in [2]^d} \epsilon_{f(x_{[d]})}\left[X_{[d]}\right] \otimes \epsilon_{x_{[d]}}\left[X_{[d]}\right]$$

Basis encodings

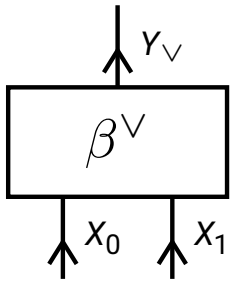
Definition (Basis encoding)

The basis encodings of a formula $f\left[X_{[d]}\right] : \{0,1\}^d \to \{0,1\}$ is the tensor

$$\beta^f\left[Y_f, X_{[d]}\right] = \sum_{x_{[d]} \in [2]^d} \epsilon_{f(x_{[d]})}\left[Y_f\right] \otimes \epsilon_{x_{[d]}}\left[X_{[d]}\right] \quad ,$$



Example: The basis encoding of the logical disjunction $\vee$ is the tensor



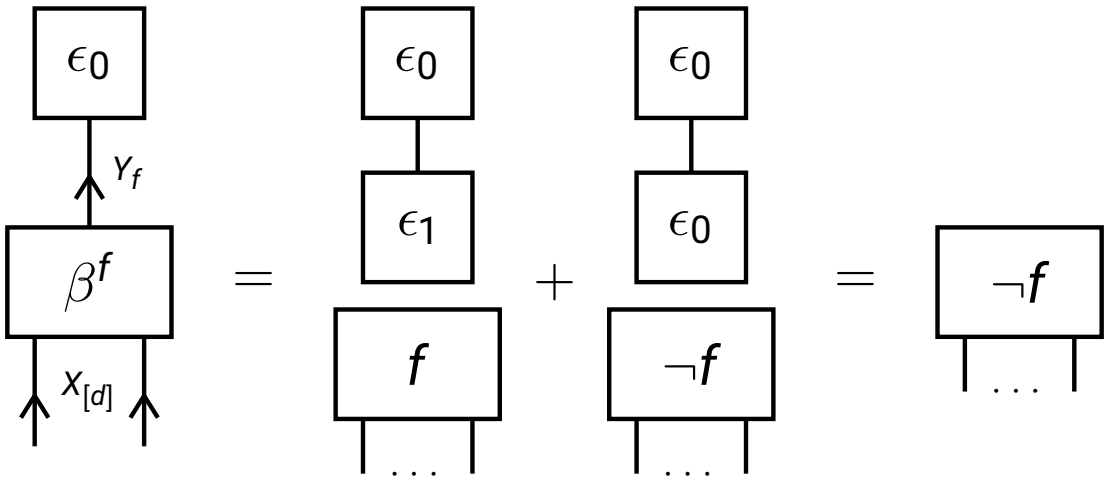
$$= \begin{array}{c} X_1 \\ \begin{array}{cc} 0 & 1 \end{array} \\ X_0 \begin{array}{c} 0 \\ 1 \end{array} \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \\ \begin{array}{cc} 0 & 1 \\ & Y_\vee \end{array} \end{array} \quad [X_0, X_1, Y_\vee]$$

Solution of the non-linearity problem

Basis encodings resolve the nonlinearity problem: Any formula $\tilde{f} \circ f$
Example: Logical negation of a formula

$$\neg f\left[X_{[d]}\right] = \left\langle \beta^f\left[Y_f, X_{[d]}\right], \neg Y_f[Y_f]\right\rangle_{\left[X_{[d]}\right]}$$

by



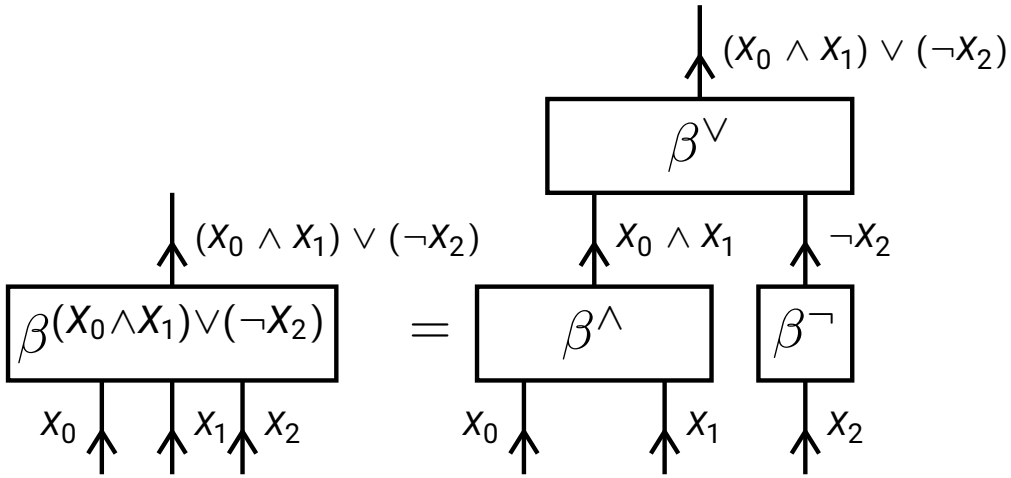
Decomposition of basis encodings

Representation of Composed Formulas

The semantics of complex formulas are retrieved by contractions of their connective semantics, which are summations of tensor coordinates among shared axes.

- Choose distributed representation to avoid contractions
- Only execute those contractions required by reasoning

Contractions can be depicted graphically by a Tensor Network:



Decomposition sparsity

Theorem

The basis encoding of a composed formula is the contraction of the basis encodings to its connectives.

Outline

- Syntactical Decomposition of Formulas
- Basis encodings
- Basis calculus
- Implementations in `tnreason`

Basis calculus perspective on logics

Boolean states are represented by one-hot encodings

$$\epsilon_{\mathrm{True}} = \epsilon_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad \text{and} \quad \epsilon_{\mathrm{False}} = \epsilon_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} .$$

The truth tables of logical connectives $\circ \in \{\wedge, \vee, \Rightarrow\}$ are represented by tensors $\beta^\circ \in \mathbb{R}^{2\times 2\times 2}$ defined as

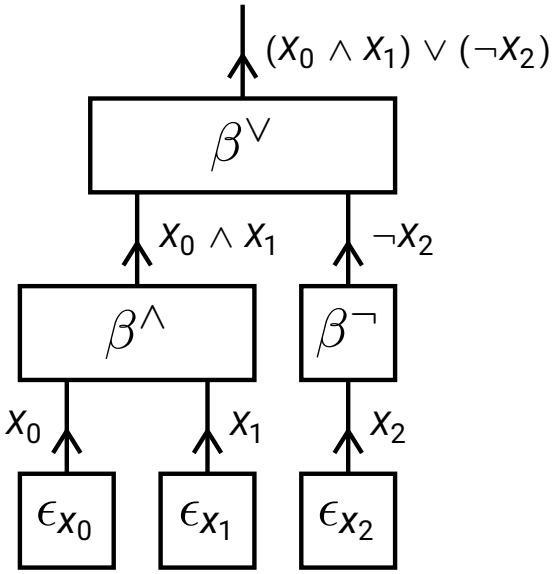
Contracting with the one-hot encoding of the inputs gives you the one-hot encoding of the formula satisfaction.

Formula evaluation by basis calculus

Basis Calculus: Computation by basis vectors

- Input states are prepared by their one-hot encodings
- Contractions with the basis encodings result in the one-hot encoding of the output

Advantage: Contractions can performed by local contractions.

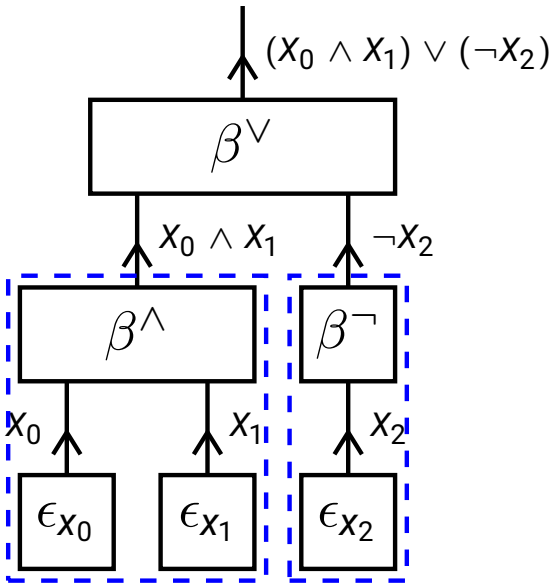


Formula evaluation by basis calculus

Basis Calculus: Computation by basis vectors

- Input states are prepared by their one-hot encodings
- Contractions with the basis encodings result in the one-hot encoding of the output

Advantage: Contractions can performed by local contractions.

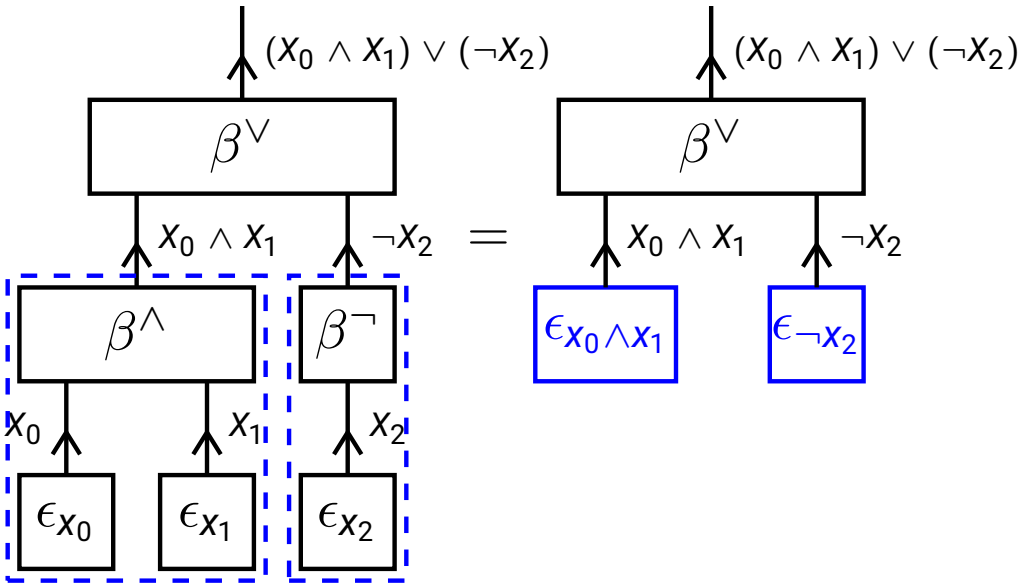


Formula evaluation by basis calculus

Basis Calculus: Computation by basis vectors

- Input states are prepared by their one-hot encodings
- Contractions with the basis encodings result in the one-hot encoding of the output

Advantage: Contractions can performed by local contractions.

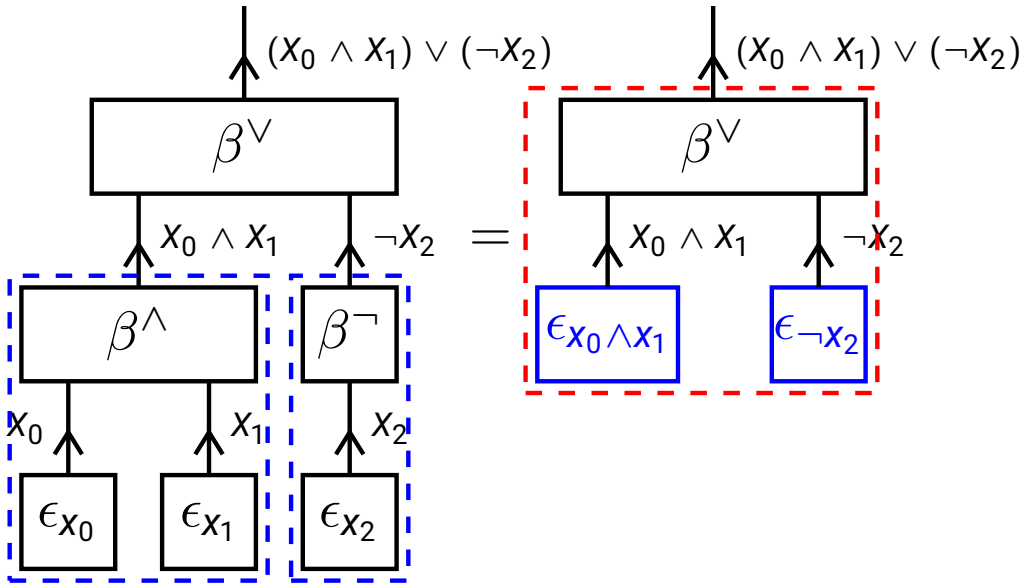


Formula evaluation by basis calculus

Basis Calculus: Computation by basis vectors

- Input states are prepared by their one-hot encodings
- Contractions with the basis encodings result in the one-hot encoding of the output

Advantage: Contractions can performed by local contractions.

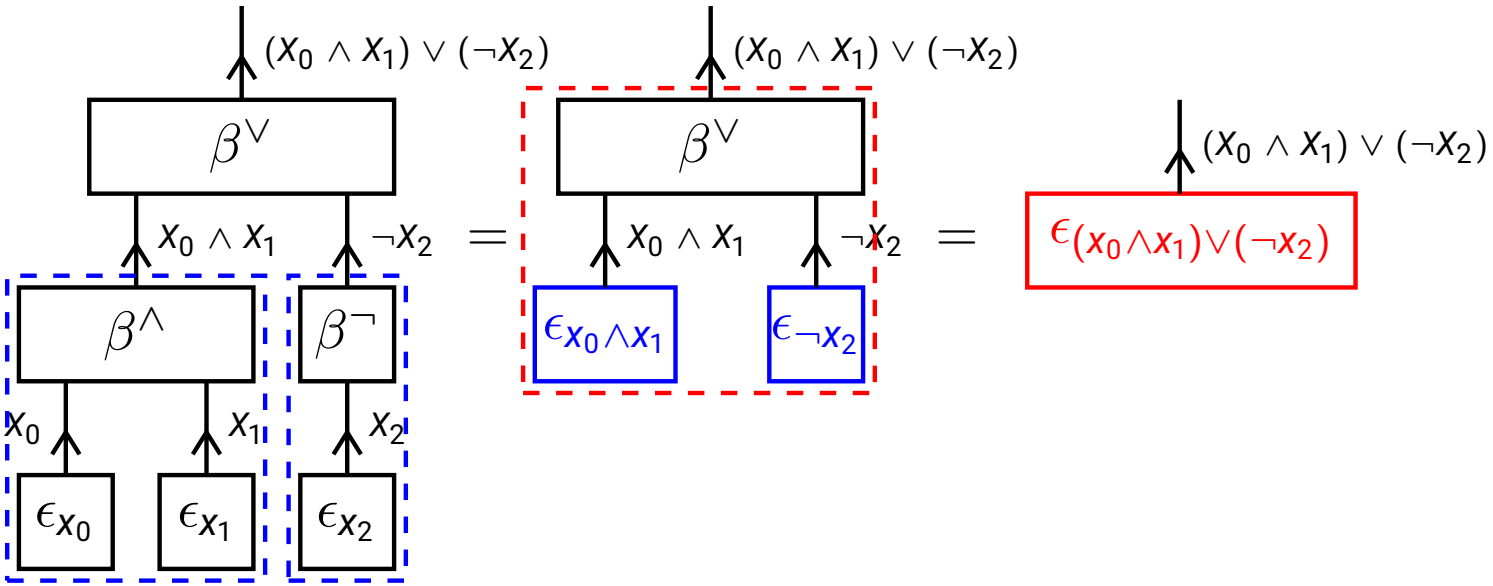


Formula evaluation by basis calculus

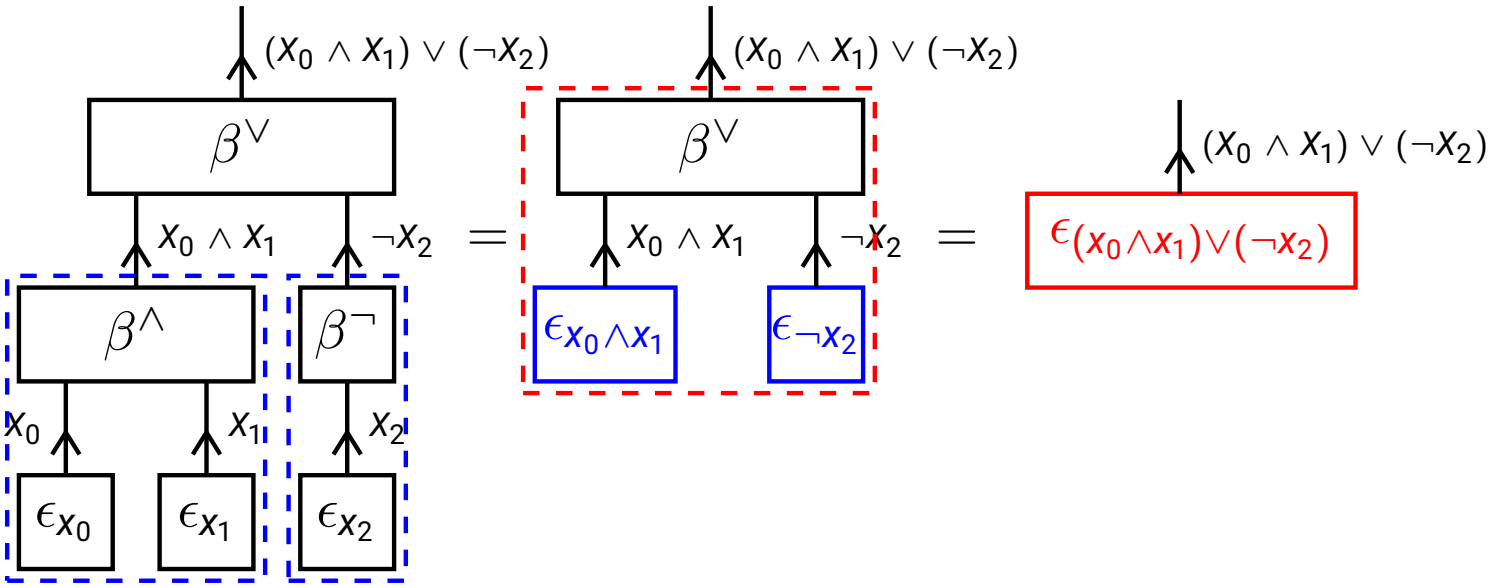
Basis Calculus: Computation by basis vectors

- Input states are prepared by their one-hot encodings
- Contractions with the basis encodings result in the one-hot encoding of the output

Advantage: Contractions can performed by local contractions.



Formula evaluation by basis calculus



This is a first example of a message passing scheme:

- "Messages" are computed by contractions of sub-networks (two blue contractions, one red contraction)
- "Passing" of previous messages (blue contractions) to compute new messages (red contraction)

Advantages of the Network Decomposition

Model-theoretic Semantics by Tensor Networks

- **Tensors encode the models** of propositional formulas by their nonzero coordinates
- **Networks eliminate redundancies** when representing similar formulas

This is an implementation of the **neural paradigm**, resulting in

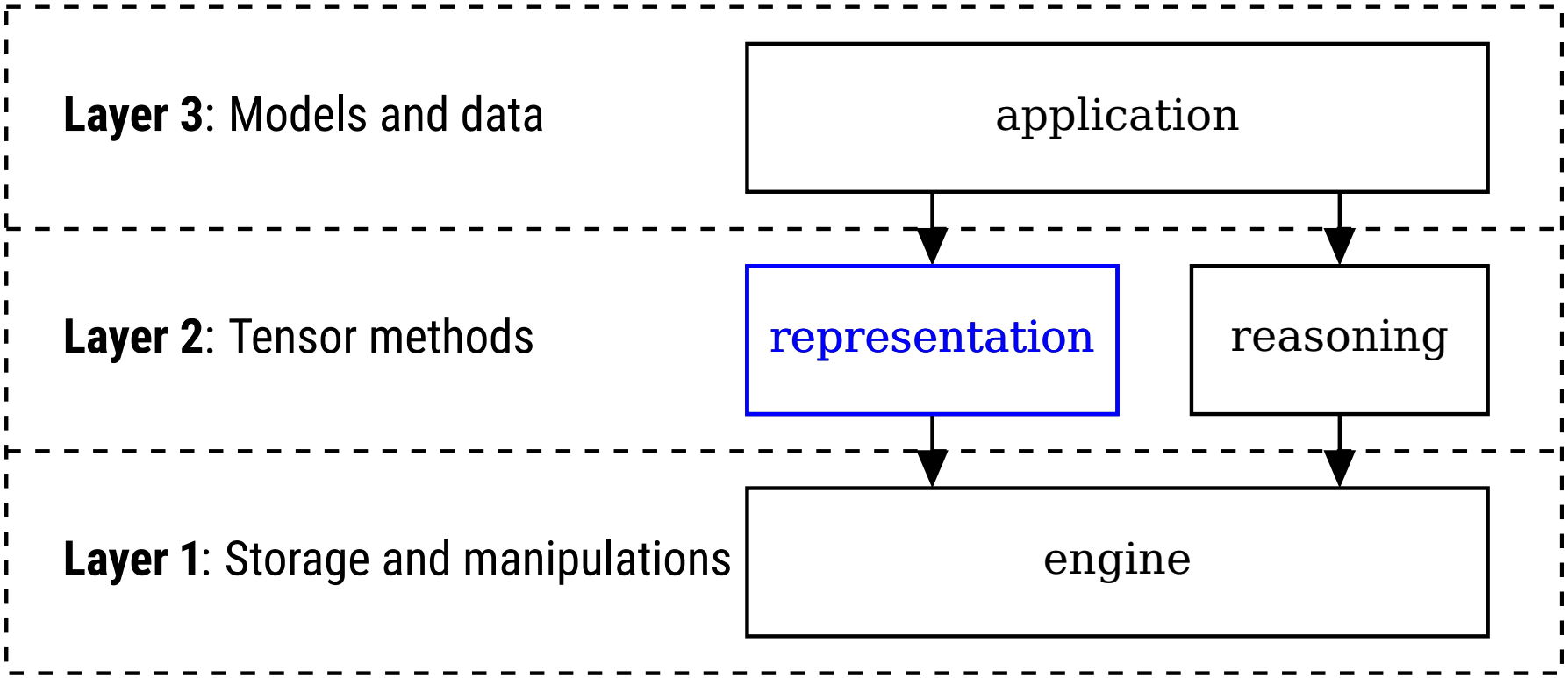
- Effective representation: Latent variables by logical formulas
- Differential parametrization: Multilinearity with respect to each core tensor

Outline

- Syntactical Decomposition of Formulas
- Basis encodings
- Basis calculus
- Implementations in `tnreason`

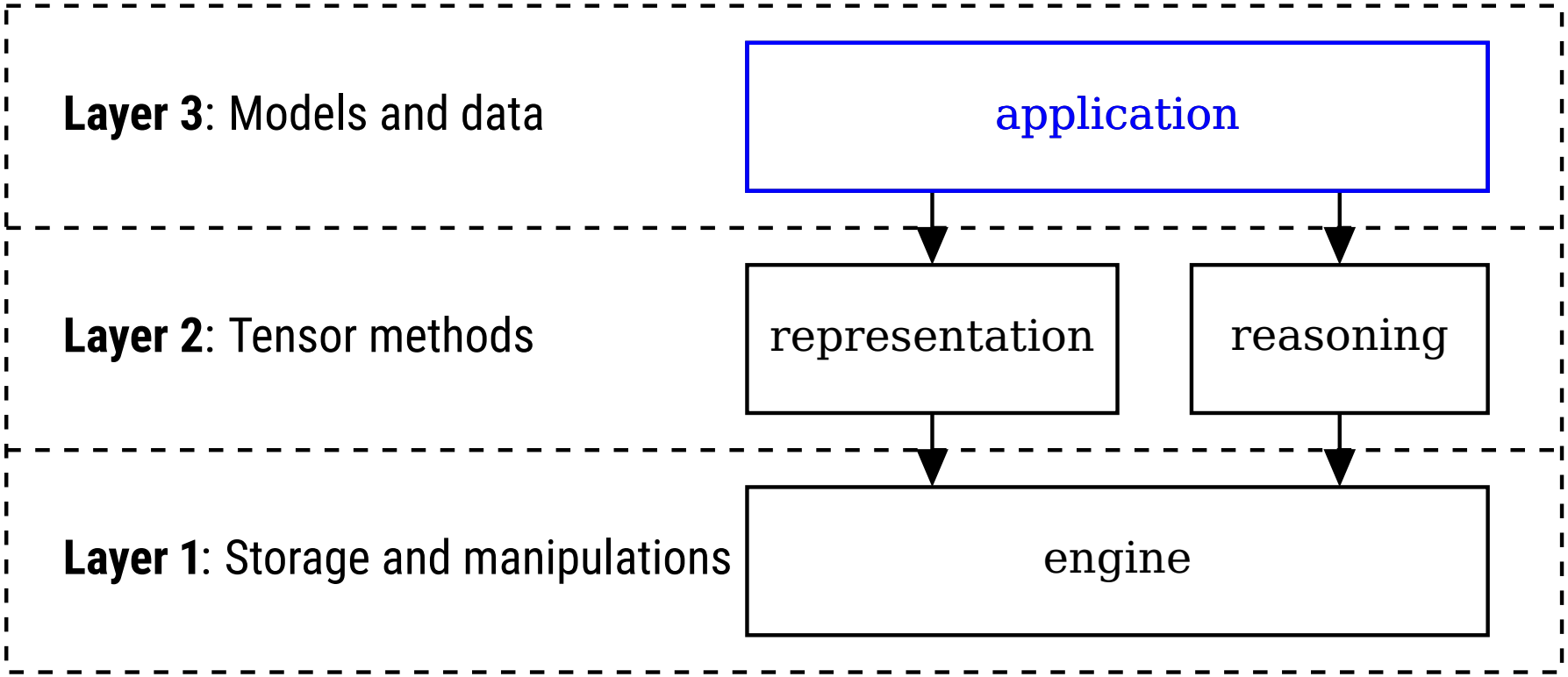
Implementations in tnreason: Subpackage representation

The subpackage representation is dedicated to the tensor encoding schemes such as basis encodings.



Implementations in tnreason: Subpackage application

The subpackage application interprets logical syntax in nested lists.



Representation of Propositional Formulas

Propositional formulas have three representations specified in the subpackage $\mathrm{representation}$:

- Syntax of formulas f is stored in a script language σ^f based on nested lists of strings
- Semantics of formulas is stored by tensor networks of connective cores
- Random variable representing the formula satisfaction

Representation of Syntax

In tnreason, propositional syntax is represented by nested lists of strings, for example:

["and", ["not", "Rained"], ["imp", "Rained", "Wet"]]

Connective have a defined representation as strings:

Unary connective $\circ$	σ°
$\neg$	*"not"*
$()$	*"id"*

Binary connective $\circ$	σ°
$\wedge$	*"and"*
$\vee$	*"or"*
$\Rightarrow$	*"imp"*
$\oplus$	*"xor"*
$\Leftrightarrow$	*"eq"*

Any other string is interpreted by an atomic formula (a so-called propositional symbol).

Representation of Syntax

Having strings representing

- Connectives (by defined representations)
- Atomic Formulas (all other strings)

we represent formulas $f_1 \circ, f_2$

$$[\sigma^{\circ}, \sigma^{f_1}, \sigma^{f_2}]$$

where we apply the conventions

- Connectives are at the 0th position in each list
- Further entries are either atoms as strings or encoded formulas itself

Examples of representations

Atomic variable Rained by

$$\sigma^{\text{Rained}} = \text{"Rained"}$$

Negative literal $\neg$Rained by

["not","Rained"]

Horn clause (Rained $\Rightarrow$ Wet) by

["imp","Rained","Wet"]

Knowledge Base ($\neg$Rained) $\wedge$ (Rained $\Rightarrow$ Wet) by

["and", ["not", "Rained"], ["imp", "Rained", "Wet"]]

Semantics

The recursive structure of the nested lists σ^f is exploited in finding tensor network representations of f. The function

encoding.create_raw_cores()

creates the connective cores to σ^f by

- When σ^f a list, create $\beta^{\sigma^f[0]}$ and add to the tensor networks created by recursive calls to the subformulas $\sigma^f[1:]$
- Return empty list, when σ^f is a string

Then we have

$$f = \left\langle \mathit{encoding.create_raw_cores}(\sigma^f)\} \cup \{\epsilon_1 \right\rangle_{[f^0,\ldots,f^{d-1}]}$$